

IMD0029 - EDB1 - 2025.2 - Unidade 3

Prof. Eiji Adachi



LEIA ANTES DE COMEÇAR

- Atividade individual sem consulta a pessoas ou materiais (impresso ou eletrônico).
 - O valor de cada questão está indicado no enunciado.
 - Mantenha celulares e outros eletrônicos desligados durante a prova.
 - Desvios éticos resultarão em nota zero nesta unidade.
 - Você recebeu diretórios para cada questão, cada um contendo um código base e um teste executável que pode ser executado via `makefile`.
△ **O teste não garante a corretude completa da implementação.**
 - **Não altere a assinatura das funções fornecidas.** Você pode criar funções auxiliares, mas a assinatura principal deve ser mantida.
-



Critérios de Correção

Serão avaliados os seguintes pontos:

- Conformidade com as **assinaturas de função** e estrutura de diretórios.
 - **Compilação limpa**, sem erros ou *warnings*.
 - **Correta execução dos programas** com os resultados esperados.
 - **Complexidade** conforme especificado no enunciado.
 - **Qualidade do código**: organização, indentação, nomes adequados, modularização, etc.
-



Entregável

- Use a mesma estrutura de diretórios recebida, com os arquivos de solução em cada pasta de questão.
- O diretório raiz deve conter seu nome em letras maiúsculas no formato `PRIMEIRO_NOME_SOBRENOME`. Exemplo:

```
JOAO_SILVA
├── lib
├── q1eq2
├── q3
└── makefile
```

- Compacte tudo em um `.zip` com o mesmo nome: `PRIMEIRO_NOME_SOBRENOME.zip`.
- ❌ Não inclua arquivos `.o` ou executáveis, mas mantenha os arquivos `makefile`. Para limpar o projeto, basta executar `make clean` na raiz do projeto, que ele automaticamente invocará o `make clean` no diretório de cada questão.

- Entregue via SIGAA até o horário estabelecido. Atrasos só serão aceitos com justificativa válida (ex.: instabilidade no SIGAA).
-

Questão 1 - Valor: 2.0

Implemente o método abaixo na classe `TabelaHash`, que representa uma **tabela hash com tratamento de colisão por encadeamento externo**:

```
void TabelaHash::inserir(std::string chave, std::string valor);
```

Comportamento esperado

- O método deve inserir o par `{chave, valor}` na tabela hash respeitando a seguinte regra: o novo elemento deve ser inserido no bucket correspondente mantendo a **ordem crescente da chave**.
- Não é permitido ordenar o bucket após a inserção — o nó deve ser inserido **diretamente na posição correta**.
- A inserção ordenada deve ser feita manipulando apenas os nós dos buckets; não é permitido usar outras estruturas de dados auxiliares, como arrays, vectors, maps, etc.
- O método retorna `true` se a inserção foi bem-sucedida e `false` caso contrário.
- Caso a chave já exista na tabela, o valor associado a essa chave deve ser atualizado com o novo valor fornecido, mantendo a ordem dos elementos no bucket. Neste caso, a função também deve retornar `true`.
- **A tabela não precisa ser redimensionada.**

Exemplo de uso

Considerando uma tabela com 5 buckets (`capacidade = 5`), e a função de espalhamento `chave % 5`:

```
TabelaHash t(5);
t.inserir(7, "A");    // chave 7 vai para bucket 2
t.inserir(12, "B");   // chave 12 vai para bucket 2
t.inserir(2, "C");    // chave 2 vai para bucket 2
t.inserir(7, "Z");    // como a chave 7 já está no bucket, o valor é
                     // atualizado
```

Estado final do bucket 2:

```
[(2, "C") -> (7, "Z") -> (12, "B")]
```

Observação: o exemplo acima usa chaves do tipo `int` apenas para facilitar o entendimento; no código, o método `inserir` recebe chaves do tipo `std::string`. Para o tipo `std::string`, é possível usar todos os operadores de comparação `==`, `<`, `>`, etc.

Onde implementar?

 Arquivo:

```
q1eq2/header/TabelaHash.h
```

 Como testar?

 Comando makefile:

```
make run-test-inserir
```

Questão 2 - Valor: 2.0

Implemente o método abaixo na classe `TabelaHash`, que representa uma **tabela hash com tratamento de colisão por encadeamento externo**:

```
bool remover(const std::string& ch);
```

Comportamento esperado

- A função deve remover da tabela o par cuja chave é igual ao parâmetro `ch`, respeitando a lógica de encadeamento externo.
- A função retorna `true` se a remoção foi bem-sucedida (ou seja, se a chave foi encontrada e removida) e `false` caso contrário.
- **A tabela não precisa ser redimensionada.**

Onde implementar?

 Arquivo:

```
q2/header/TabelaHash.h
```

Como testar?

 Comando makefile:

```
make run-test-remover
```

Questão 3 - Valor: 2.0

Implemente o método abaixo na classe `TabelaHash`, que representa uma **tabela hash com endereçamento aberto usando sondagem linear**:

```
bool TabelaHash::inserir(std::string chave, std::string valor);
```

Comportamento esperado

- A função deve inserir o par `{chave, valor}` na tabela, respeitando a lógica de endereçamento aberto usando sondagem linear em caso de colisão.
- **A tabela não precisa ser redimensionada.**
- Caso a tabela esteja cheia, deve lançar exceção do tipo `std::overflow_error`. Para isso, basta usar a instrução `throw std::overflow_error("Tabela cheia.");`.

Onde implementar?

 Arquivo:

```
q3/header/TabelaHash.h
```

Como testar?

 Comando makefile:

```
make run-test-inserir
```